

بخش اول

پیاده‌سازی قوانین اصلی :

```
130
131 def evolve(self):
132     if self.fastMode:
133         self.update_grid_fast()
134         return
135
136     new_grid = self.grid.copy()
137     for i in range(self.rows):
138         for j in range(self.cols):
139             alive_neighbors = 0
140             for r in range(-1, 2):
141                 for c in range(-1, 2):
142                     if r == 0 and c == 0:
143                         continue
144
145                     ni = i + r
146                     nj = j + c
147
148                     if self.finite:
149                         if ni < 0 or ni >= self.rows or nj < 0 or nj >= self.cols:
150                             continue
151                     else:
152                         ni %= self.rows
153                         nj %= self.cols
154
155                     if self.grid[ni, nj] == self.aliveValue:
156                         alive_neighbors += 1
157
158             if self.grid[i, j] == self.aliveValue:
159                 if alive_neighbors < 2 or alive_neighbors > 3:
160                     new_grid[i, j] = self.deadValue
161                 else:
162                     new_grid[i, j] = self.aliveValue
163             else:
164                 if alive_neighbors == 3:
165                     new_grid[i, j] = self.aliveValue
166                 else:
167                     new_grid[i, j] = self.deadValue
168
169     self.grid = new_grid
170
```

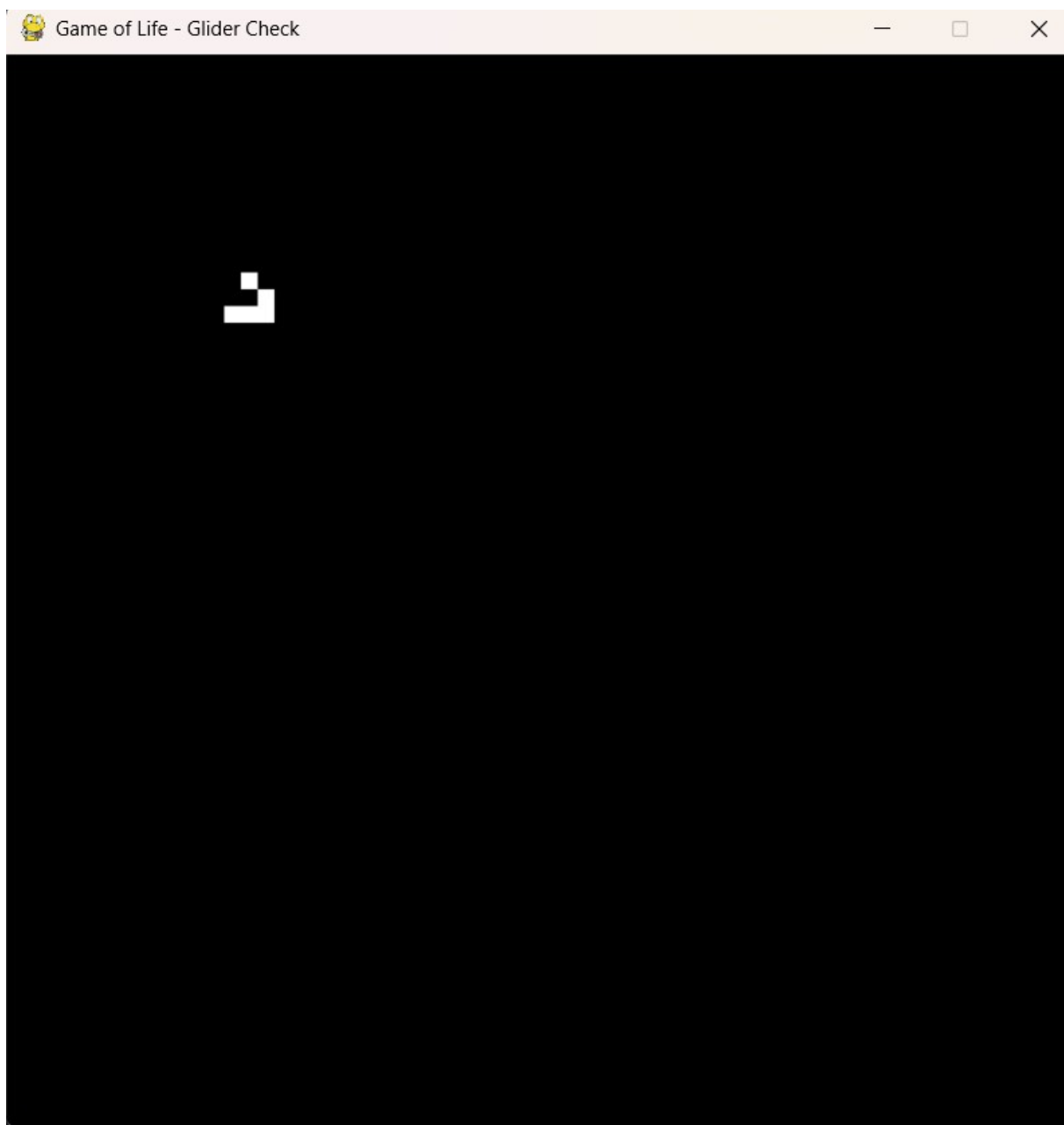
در متد evolve ابتدا بررسی شده که اگر حالت fastMode فعال باشد، به روزرسانی شبکه از طریق تابع جداگانه‌ی update_grid_fast انجام شود و ادامه‌ی متد اجرا نشود. در غیر این صورت، برنامه وارد حالت معمولی محاسبه می‌شود. برای اینکه تغییرات نسل جدید روی محاسبات نسل فعلی اثر نگذارد، ابتدا یک کپی از شبکه فعلی در متغیر new_grid گرفته شده است.

بعد از آن، کد با دو حلقه روی تمام سطرها و ستون‌های شبکه حرکت می‌کند و هر سلول را جداگانه بررسی می‌کند. برای هر سلول، متغیر alive_neighbors تعداد همسایه‌های زنده را نگه می‌دارد. سپس با دو حلقه‌ی دیگر، خانه‌های اطراف سلول در بازه‌ی یک خانه‌ای بررسی می‌شوند. حالت $r = 0$ و $c = 0$ کنار گذاشته شده، چون این حالت مربوط به خود سلول است و نباید به عنوان همسایه شمرده شود.

در ادامه، مختصات همسایه با اضافه کردن r و c به مختصات سلول فعلی ساخته می‌شود. اگر شبکه محدود باشد، همسایه‌هایی که مختصاتشان خارج از محدوده‌ی شبکه است نادیده گرفته می‌شوند. اما اگر شبکه محدود نباشد، با استفاده از عملگر $\%$ مختصات همسایه‌ها به صورت حلقوی اصلاح می‌شود تا سلول‌های لبه‌ی شبکه به سمت مقابل وصل شوند. سپس اگر مقدار آن خانه برابر با aliveValue باشد، تعداد همسایه‌های زنده یکی زیاد می‌شود.

بعد از شمارش همسایه‌ها، وضعیت سلول فعلی تعیین می‌شود. اگر سلول زنده باشد، فقط در حالتی زنده می‌ماند که تعداد همسایه‌های زنده‌اش ۲ یا ۳ باشد؛ در غیر این صورت مقدار آن در new_grid برابر با deadValue قرار می‌گیرد. اگر سلول مرده باشد، فقط زمانی زنده می‌شود که دقیقاً ۳ همسایه‌ی زنده داشته باشد. در پایان، بعد از اینکه وضعیت همه‌ی سلول‌ها در new_grid مشخص شد، مقدار self.grid با new_grid جایگزین می‌شود تا شبکه وارد نسل بعدی شود.

شبیه‌سازی گلایدر:



برای بخش شبیه‌سازی گلایدر، هدف این بود که رفتار یکی از ساده‌ترین و معروف‌ترین الگوهای متحرک در بازی زندگی کانوی بررسی شود. گلایدر از چند سلول زنده تشکیل شده، اما نکته جالبش این است که مثل الگوهای ثابت سر جای خودش نمی‌ماند. وقتی شبیه‌سازی اجرا می‌شود، گلایدر در چند نسل پشت سر هم تغییر شکل می‌دهد و بعد از هر چهار مرحله دوباره به شکل اولیه‌اش برمی‌گردد، با این تفاوت که یک خانه به صورت مورب جلو رفته است. به همین دلیل گفته می‌شود سرعت حرکت آن $C/4$ است.

چیزی که گلايدر را مهم می کند فقط حرکت کردنش نیست، بلکه این است که می تواند مثل یک پیام یا سیگنال در صفحه حرکت کند. یعنی اگر در یک مسیر مشخص گلايدر وجود داشته باشد، می توانیم آن را به عنوان بیت ۱ در نظر بگیریم و اگر گلايدری وجود نداشته باشد، آن را بیت ۰ حساب کنیم. به این ترتیب، گلايدرها می توانند نقش انتقال اطلاعات را در دنیای بازی زندگی داشته باشند.

از طرفی گلايدرها وقتی به هم یا به الگوهای دیگر برخورد می کنند، رفتارهای مختلفی ایجاد می شود؛ مثلاً ممکن است از بین بروند، مسیرشان تغییر کند یا یک ساختار جدید بسازند. همین برخوردها باعث می شود بتوان از گلايدرها برای ساختن مدارها و گیت های منطقی استفاده کرد. اگر زمان و مسیر حرکت گلايدرها درست تنظیم شود، برخورد آنها می تواند مثل یک عملیات منطقی عمل کند.

در کل، در این قسمت دیدیم که گلايدر با اینکه یک الگوی خیلی کوچک و ساده است، می تواند نقش مهمی در بازی زندگی کانوی داشته باشد. این الگو نشان می دهد که چطور از چند قانون ساده، می شود چیزی شبیه انتقال سیگنال، ارتباط و حتی محاسبه ساخت.

پیاده‌سازی پارسر فایل‌های الگو:

تابع `parse_pattern`: این تابع نقطه‌ی ورود اصلی است. ابتدا با بررسی پسوند فایل تصمیم می‌گیرد که کدام پارسر اجرا شود؛ اگر نام فایل به `rle` ختم شود، کار را به تابع `rle_parse` می‌سپارد و در غیر این صورت خودش فایل را به صورت `Plaintext` پردازش می‌کند.

در حالت `Plaintext`، تمام خطوط فایل خوانده شده و پس از حذف فاصله‌های اضافی، خطوط توضیحی که با علامت `!` شروع می‌شوند نادیده گرفته می‌شوند. سپس عرض الگو برابر با طول بلندترین خط و ارتفاع برابر با تعداد خطوط در نظر گرفته می‌شود. بعد از آن، تمام کاراکترها پیمایش می‌شوند و هر جا کاراکتر `O` دیده شود، مختصات آن سلول به عنوان یک سلول زنده در لیست `live_cells` ذخیره می‌گردد. در نهایت تابع یک سه‌تایی شامل عرض، ارتفاع و لیست مختصات سلول‌های زنده برمی‌گرداند.

تابع `rle_parse`: این تابع مخصوص فرمت فشرده `RLE` است. ابتدا تمام خطوط فایل خوانده شده و خطوطی که با `#` (توضیحات و متادیتا) یا `!` شروع می‌شوند حذف می‌شوند. سپس اولین خط باقی‌مانده به عنوان هدر (`Header`) در نظر گرفته می‌شود. در این هدر، ابعاد الگو با کمک عبارات باقاعده (`Regular Expression`) و الگوهای `X = ...` و `Y = ...` استخراج می‌شوند تا مقدار عرض و ارتفاع مشخص شود. اگر این ابعاد پیدا نشوند، تابع خطا می‌دهد تا از خواندن فایل نامعتبر جلوگیری شود. بعد از هدر، بقیه‌ی خطوط به هم چسبانده می‌شوند تا رشته‌ی داده‌ی اصلی به دست بیاید. سپس این رشته کاراکتر به کاراکتر پیمایش می‌شود.

منطق فرمت `RLE` این‌طور است که هر کاراکتر می‌تواند یک عدد تکرار (`run count`) قبل از خودش داشته باشد. برای همین، هر وقت رقمی دیده شود در متغیر `run_count` جمع می‌شود، و وقتی به یک کاراکتر معنادار برسیم، آن عدد به عنوان تعداد تکرار استفاده می‌شود (و اگر عددی نبود، پیش‌فرض ۱ در نظر گرفته می‌شود).

کاراکترهای معنادار این‌ها هستند:

- `O`: سلول زنده. به تعداد `count` سلول زنده در همان سطر اضافه می‌شود و ستون جلو می‌رود.
- `b`: سلول مرده. فقط شمارنده‌ی ستون به اندازه‌ی `count` جلو می‌رود.
- `$`: پایان سطر. شمارنده‌ی سطر به اندازه‌ی `count` جلو می‌رود و ستون به صفر برمی‌گردد. `!`: پایان کل الگو که با رسیدن به آن حلقه متوقف می‌شود.

در پایان، این تابع هم مانند تابع قبلی یک سه‌تایی شامل عرض، ارتفاع و لیست مختصات سلول‌های زنده برمی‌گرداند. در مجموع، این دو تابع در کنار هم امکان لود الگوهای استاندارد `LifeWiki` را فراهم می‌کنند؛ الگوهای ساده و کوچک از طریق فرمت `Plaintext` و الگوهای بزرگ و پیچیده به صورت فشرده از طریق فرمت `RLE`. خروجی یکسان هر دو تابع (عرض، ارتفاع و لیست سلول‌های زنده) باعث می‌شود بقیه‌ی برنامه بدون توجه به نوع فایل ورودی، بتواند الگو را به شکل یکنواخت داخل شبکه قرار دهد.

بهینه‌سازی محاسباتی با کانولوشن:

```
117 def update_grid_fast(self):
118     if self.finite:
119         conv_grid = ndimage.convolve(self.grid, self.neighborhood, mode="constant", cval=0)
120     else:
121         conv_grid = ndimage.convolve(self.grid, self.neighborhood, mode="wrap")
122
123     next_board = ((self.grid == self.aliveValue) & (conv_grid >= 2) & (conv_grid <= 3)) | ((self.grid == self.deadValue) & (conv_grid == 3))
124
125     self.grid = next_board.astype(self.grid.dtype) * self.aliveValue
126
```

در این بخش، متد `update_grid_fast` برای به‌روزرسانی سریع شبکه‌ی بازی زندگی کانوی پیاده‌سازی شد. هدف این متد، جایگزین کردن روش سلول‌به‌سلول با یک روش برداری است تا شمارش همسایه‌های تمام سلول‌ها توسط توابع بهینه‌ی `SciPy` و `NumPy` انجام شود. در این روش، شبکه‌ی فعلی در `self.grid` به‌صورت یک آرایه‌ی دوبعدی `NumPy` نگهداری می‌شود که مقدار ۱ در آن نشان‌دهنده‌ی سلول زنده و مقدار ۰ نشان‌دهنده‌ی سلول مرده است.

برای شمارش همسایه‌های زنده از تابع `ndimage.convolve` استفاده شده است. `ndimage` یکی از ماژول‌های کتابخانه‌ی `SciPy` برای انجام پردازش‌های مبتنی بر آرایه و تصویر است و تابع `convolve` در آن، یک الگوی کوچک را روی تمام نقاط یک آرایه‌ی دوبعدی اعمال می‌کند. ورودی اول این تابع، یعنی `self.grid`، همان شبکه‌ی فعلی بازی است. ورودی دوم، یعنی `self.neighborhood`، الگوی همسایگی یا کرنل 3×3 زیر است:

```

1 1 1
0 1 0
1 1 1
```

مقدارهای یک در این الگو مشخص می‌کنند که خانه‌های اطراف سلول باید در شمارش وارد شوند و مقدار صفر وسط مشخص می‌کند که خود سلول فعلی نباید شمرده شود. تابع `ndimage.convolve` این الگو را روی هر سلول از شبکه قرار می‌دهد. در هر موقعیت، مقدار هر خانه‌ی شبکه در مقدار متناظر خود در کرنل ضرب می‌شود و سپس تمام نتایج با هم جمع می‌شوند. بنابراین اگر یک خانه‌ی اطراف زنده باشد، مقدار 1×1 در جمع وارد می‌شود و اگر مرده باشد، مقدار 0×1 وارد جمع می‌شود. مقدار سلول مرکزی نیز، حتی اگر زنده باشد، به دلیل ضرب شدن در صفر وارد جمع نمی‌شود.

برای مثال، اگر یک سلول سه همسایه‌ی زنده داشته باشد، نتیجه‌ی کانولوشن در موقعیت همان سلول برابر با ۳ می‌شود. این فرآیند برای همه‌ی سلول‌های شبکه تکرار می‌شود و خروجی تابع در متغیر `conv_grid` ذخیره می‌گردد. `conv_grid` نیز یک آرایه‌ی دوبعدی با همان ابعاد `self.grid` است، اما تفاوت آن با شبکه‌ی اصلی در این است که هر مقدار در آن تعداد همسایه‌های زنده‌ی سلول متناظر را نشان می‌دهد، نه وضعیت زنده یا مرده بودن خود آن سلول.

در فراخوانی تابع کانولوشن، علاوه بر شبکه و کرنل، نحوه‌ی برخورد با مرزهای شبکه نیز تعیین شده است. اگر `self.finite` برابر با `True` باشد، از حالت `mode="constant"` همراه با `cval=0` استفاده می‌شود. در این حالت، بخش‌هایی از کرنل که هنگام قرار گرفتن روی

لبه‌ها از شبکه خارج می‌شوند، مقدار صفر یا سلول مرده در نظر گرفته می‌شوند. بنابراین در شبکه‌ی محدود، سلول‌های خارج از مرز وجود ندارند و در شمارش همسایه‌ها اثری ندارند.

اگر `self.finite` برابر با `False` باشد، از حالت `mode="wrap"` استفاده می‌شود. این حالت شبکه را حلقوی در نظر می‌گیرد؛ یعنی اگر کرنل از سمت راست شبکه خارج شود، ادامه‌ی آن از سمت چپ شبکه خوانده می‌شود و همین حالت برای بالا و پایین شبکه نیز برقرار است. در نتیجه، مرزهای روبه‌روی شبکه به هم متصل می‌شوند و جهان بازی حالت توروئیدال پیدا می‌کند.

پس از به‌دست آمدن `conv_grid`، قوانین بازی زندگی کانوی با استفاده از عملیات برداری NumPy اعمال می‌شوند. عبارت `self.grid` `self.aliveValue ==` آرایه‌ای بولی می‌سازد که در آن خانه‌های زنده مقدار `True` دارند. عبارت‌های `conv_grid >= 2` و `conv_grid <= 3` نیز مشخص می‌کنند که کدام خانه‌ها دو یا سه همسایه‌ی زنده دارند. با ترکیب این شرط‌ها به‌وسیله‌ی عملگر `&`، وضعیت بقای سلول‌های زنده مشخص می‌شود؛ یعنی سلولی که در حال حاضر زنده است و دو یا سه همسایه‌ی زنده دارد، در نسل بعد نیز زنده باقی می‌ماند.

بخش دوم شرط، یعنی `self.grid == self.deadValue` و `conv_grid == 3`، قانون تولیدمثل را پیاده‌سازی می‌کند. این قسمت مشخص می‌کند که هر سلول مرده‌ای که دقیقاً سه همسایه‌ی زنده داشته باشد، در نسل بعد زنده شود. در نهایت، دو شرط بقا و تولیدمثل با عملگر `|` که به معنای «یا» است ترکیب می‌شوند و نتیجه در `next_board` قرار می‌گیرد. این متغیر یک آرایه‌ی بولی است که وضعیت نسل بعدی را با مقادیر `True` و `False` مشخص می‌کند.

در مرحله‌ی آخر، با استفاده از `astype(self.grid.dtype)`، مقادیر بولی به نوع داده‌ی شبکه تبدیل می‌شوند؛ یعنی `True` به مقدار ۱ و `False` به مقدار ۰ تبدیل می‌شود. سپس این آرایه در `self.aliveValue` ضرب شده و جایگزین `self.grid` می‌شود. در نتیجه، وضعیت شبکه وارد نسل بعدی خواهد شد.

این روش از نظر مرتبه‌ی زمانی همچنان $\Theta(N^2)$ است، زیرا برای یک شبکه‌ی $N \times N$ باید اطلاعات همه‌ی N^2 سلول بررسی شود. با این حال، در عمل از روش حلقه‌ای سریع‌تر است، زیرا حلقه‌های مربوط به کانولوشن و شرط‌های آرایه‌ای در کدهای بهینه‌ی داخلی SciPy و NumPy اجرا می‌شوند و نیازی نیست Python برای هر سلول و هر همسایه به‌صورت جداگانه حلقه اجرا کند.

بخش دوم

پیاده‌سازی پایه مورچه:

در این بخش، کلاس `LangtonsAnt` برای شبیه‌سازی مورچه لنگتون روی یک شبکه‌ی دوبعدی پیاده‌سازی شد. این کلاس وضعیت خانه‌های شبکه، موقعیت و جهت فعلی مورچه و قوانین حرکت را نگهداری می‌کند. در هر مرحله، مورچه رنگ خانه‌ی فعلی را تغییر می‌دهد، به راست یا چپ می‌چرخد و سپس یک خانه به جلو حرکت می‌کند.

```
13
14 def __init__(self, N, ant_position, rules):
15     """
16     Initialize the Langton's Ant simulation.
17
18     Args:
19         N (int): The grid size (NxN).
20         ant_position (tuple): Starting coordinate of the ant as (r, c).
21         rules (dict or str): Dictionary mapping {color: (next_color, 'R'/'L')}
22                             OR a ruleset string like "RLR", "LLRR", "LARRRRRLLR".
23     """
24     self.grid = np.zeros((N, N), np.uint8)
25     self.N = N
26     self.ant_position = ant_position
27     self.current_dir = "U" # Possible directions: 'U', 'R', 'D', 'L'
28
29     if isinstance(rules, str):
30         self.rules = {}
31         num_colors = len(rules)
32         for i, turn in enumerate(rules):
33             next_color = (i + 1) % num_colors
34             self.rules[i] = (next_color, turn.upper())
35     else:
36         self.rules = rules
37
```

در سازنده‌ی کلاس، یک شبکه‌ی $N \times N$ با استفاده از `np.zeros` ساخته می‌شود. تمام خانه‌های این شبکه در ابتدا دارای مقدار صفر هستند. نوع داده‌ی `uint8` نیز برای ذخیره‌ی حالت خانه‌ها انتخاب شده، زیرا مقادیر رنگ‌ها اعداد صحیح کوچک هستند و به فضای زیادی نیاز ندارند.

اندازه‌ی شبکه در `self.N`، موقعیت اولیه‌ی مورچه در `self.ant_position` و جهت اولیه‌ی آن در `self.current_dir` ذخیره می‌شود. جهت اولیه‌ی مورچه `U` در نظر گرفته شده است؛ یعنی مورچه در ابتدای شبیه‌سازی به سمت بالا قرار دارد. قوانین حرکت نیز در `self.rules` ذخیره می‌شوند تا در هر مرحله، رفتار مورچه بر اساس رنگ خانه‌ی فعلی مشخص شود.


```

38     def get_states(self):
39         """
40         Returns the current state grid of the cells.
41         """
42         return self.grid
43
44     def get_current_position(self):
45         """
46         Returns the ant's current position as a tuple (r, c).
47         """
48         return self.ant_position
49

```

این متد وضعیت فعلی شبکه را برمی گرداند. خروجی آن همان آرایه‌ی دوبعدی `self.grid` است که رنگ یا حالت تمام خانه‌ها را نگهداری می‌کند. نمایشگر Pygame از این متد برای رسم شبکه در هر مرحله استفاده می‌کند.

متد `get_current_position` نیز مختصات فعلی مورچه را به صورت (r, c) برمی گرداند. مقدار r نشان‌دهنده‌ی شماره‌ی سطر و مقدار c نشان‌دهنده‌ی شماره‌ی ستون است. این مختصات برای مشخص کردن محل مورچه روی صفحه‌ی گرافیکی استفاده می‌شود.

```

49
50     def step(self):
51         """
52         Perform a single simulation step following the multi-color ruleset.
53         """
54         r, c = self.ant_position
55         current_color = self.grid[r, c]
56
57         # Retrieve next color state and turn direction
58         next_color, turn_direction = self.rules[current_color]
59
60         # Toggle/update color of the current square
61         self.grid[r, c] = next_color
62
63         # 1. Update ant orientation (turn R or L)
64         dirs_clockwise = ["U", "R", "D", "L"]
65         curr_idx = dirs_clockwise.index(self.current_dir)
66
67         if turn_direction == "R":
68             self.current_dir = dirs_clockwise[(curr_idx + 1) % 4]
69         elif turn_direction == "L":
70             self.current_dir = dirs_clockwise[(curr_idx - 1) % 4]
71
72         # 2. Move forward one unit in the new direction
73         if self.current_dir == "U":
74             r -= 1
75         elif self.current_dir == "D":
76             r += 1
77         elif self.current_dir == "R":
78             c += 1
79         elif self.current_dir == "L":
80             c -= 1
81
82         # Apply toroidal wrapping
83         self.ant_position = (r % self.N, c % self.N)

```

در ابتدای متد `step`، موقعیت فعلی مورچه خوانده شده و رنگ خانه‌ای که مورچه روی آن قرار دارد از شبکه دریافت می‌شود. سپس با استفاده از رنگ فعلی، قانون متناظر از دیکشنری `self.rules` گرفته می‌شود. این قانون شامل رنگ بعدی خانه و جهت چرخش مورچه است. پس از خواندن قانون، رنگ خانه‌ی فعلی به مقدار جدید تغییر می‌کند.

در قانون کلاسیک، اگر مورچه روی خانه‌ی سفید باشد، خانه سیاه شده و مورچه به راست می‌چرخد. اگر روی خانه‌ی سیاه باشد، رنگ خانه به سفید تغییر کرده و مورچه به چپ می‌چرخد.

برای نگهداری جهت‌ها، چهار جهت `U`، `R`، `D` و `L` به ترتیب ساعت‌گرد در یک لیست قرار گرفته‌اند. ابتدا موقعیت جهت فعلی مورچه در این لیست پیدا می‌شود. اگر قانون `R` باشد، جهت مورچه یک مرحله در جهت ساعت‌گرد تغییر می‌کند و اگر قانون `L` باشد، یک مرحله در خلاف جهت ساعت‌گرد تغییر می‌کند.

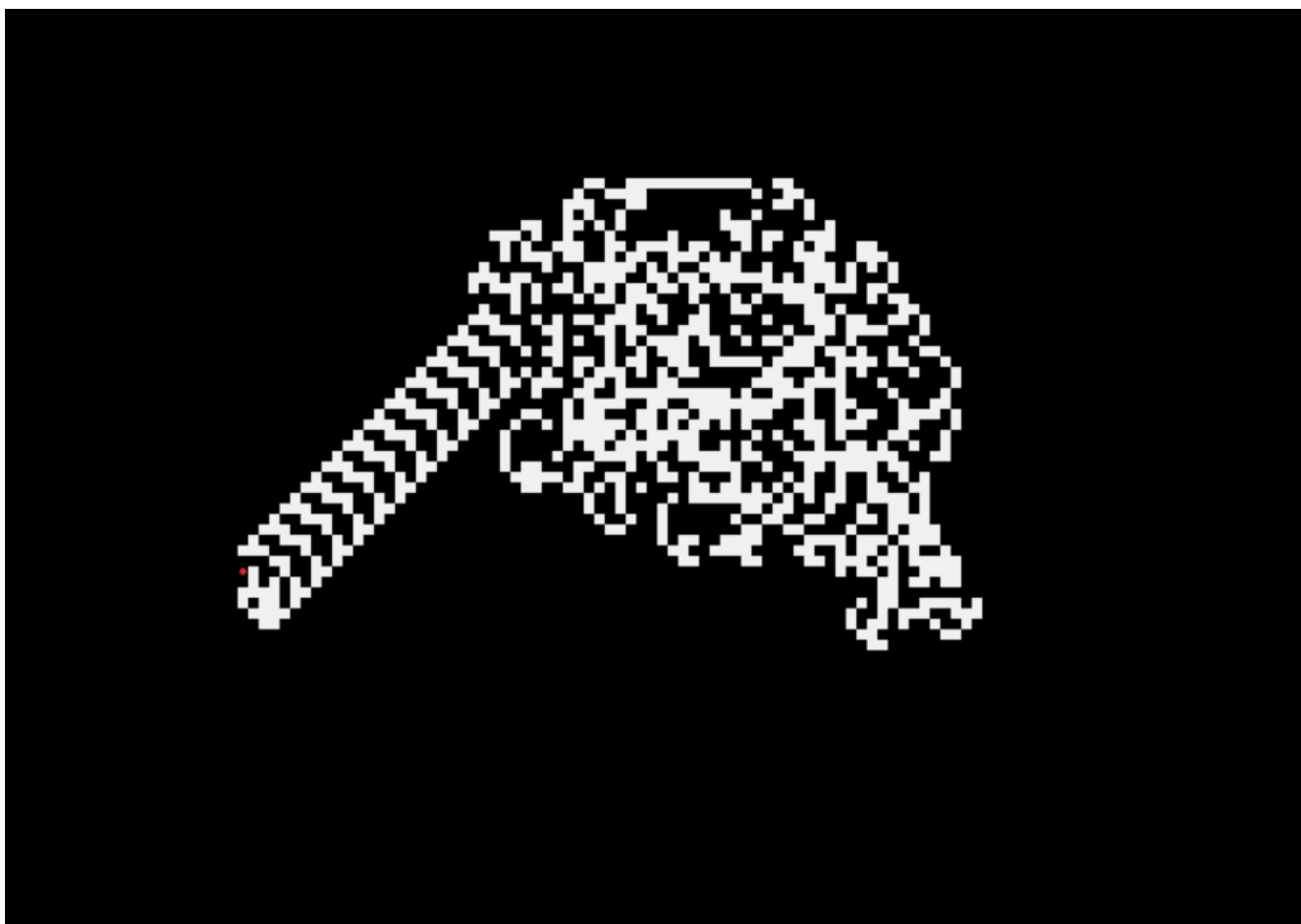
استفاده از عملگر باقیمانده بر ۴ باعث می شود چرخش در ابتدا و انتهای لیست نیز درست انجام شود. برای مثال، چرخش به چپ از جهت بالا، مورچه را به سمت چپ هدایت می کند.

پس از تغییر جهت، مورچه یک خانه در جهت جدید حرکت می کند. در حرکت به بالا یا پایین، مقدار سطر تغییر می کند و در حرکت به راست یا چپ، مقدار ستون تغییر می یابد. به این ترتیب، مختصات جدید مورچه بر اساس جهت فعلی آن محاسبه می شود.

در پایان متد **step**، مختصات جدید مورچه با استفاده از عملگر باقیمانده نسبت به اندازه ی شبکه محاسبه می شود. این کار باعث می شود شبکه حالت حلقوی یا توروئیدال داشته باشد. بنابراین اگر مورچه از یکی از لبه های شبکه خارج شود، از لبه ی مقابل وارد خواهد شد و موقعیت آن همیشه داخل محدوده ی آرایه باقی می ماند.

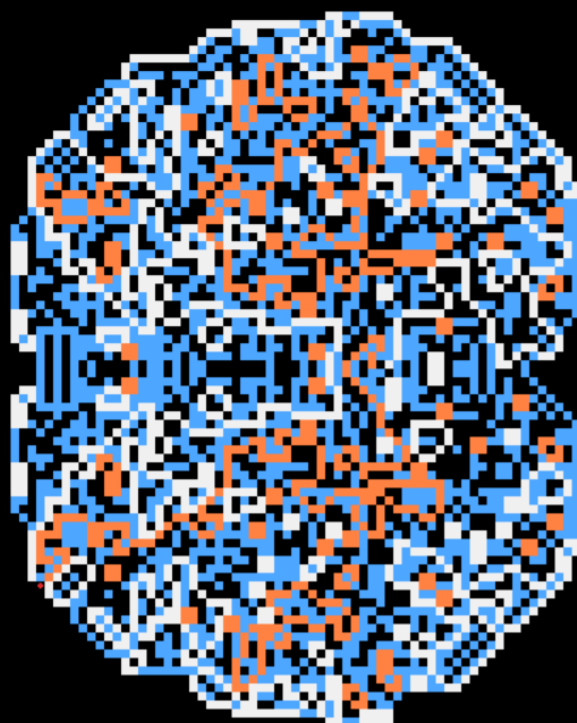
در انتها نیز متد **update** تنها متد **step** را فراخوانی می کند و باعث می شود شبیه سازی یک مرحله جلو برود. این متد به عنوان یک نام جایگزین برای **step** در نظر گرفته شده تا کلاس بتواند به سادگی در حلقه های انیمیشن و نمایش گرافیکی استفاده شود.

شبیه‌سازی حرکت مورچه بعد از حدود ده هزار گام:

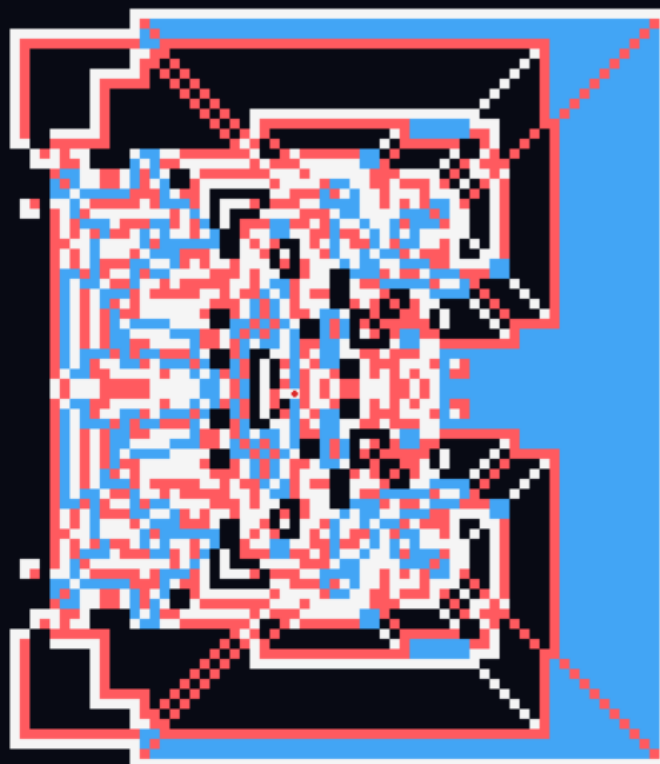


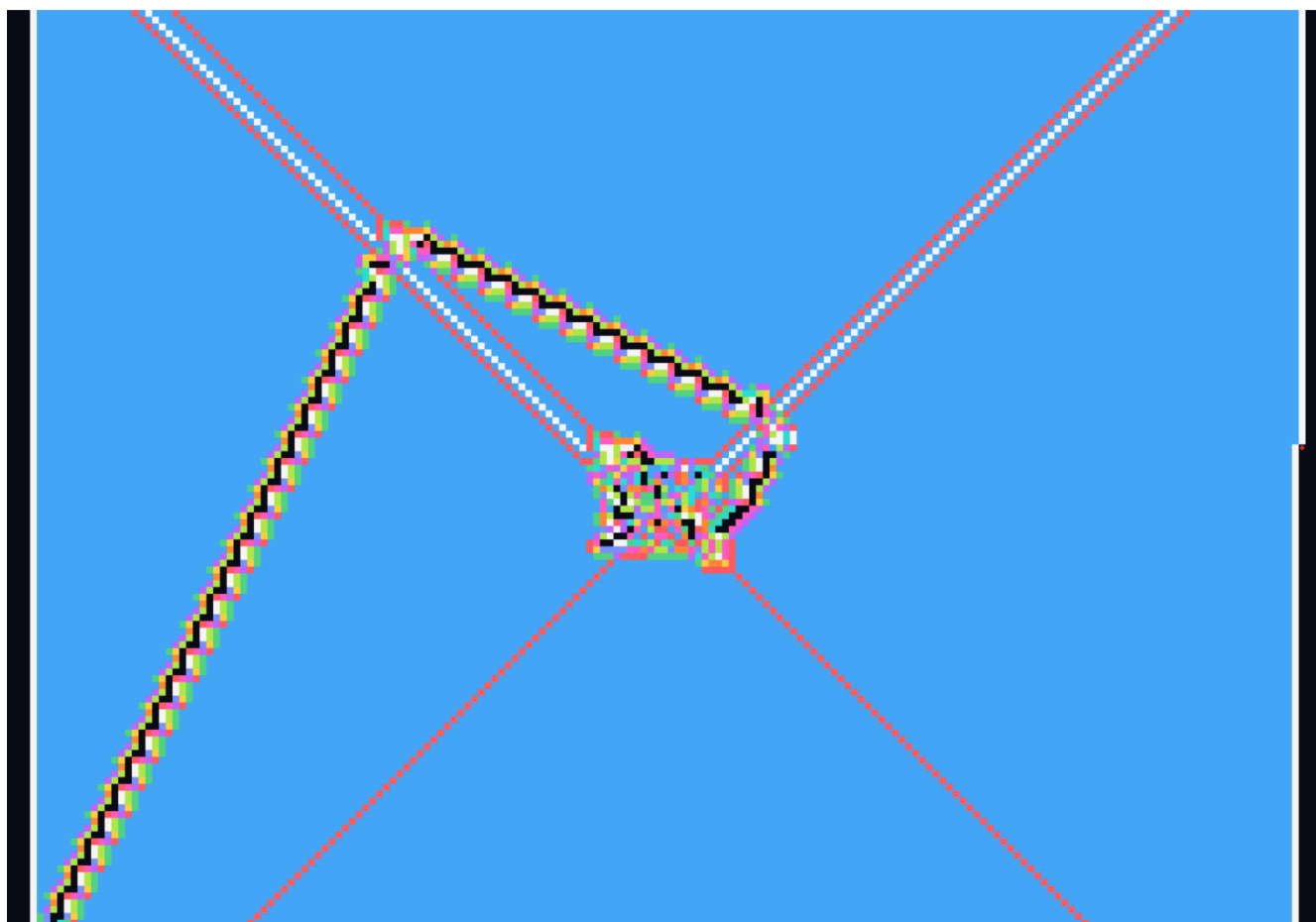
بهبود مورچه با پشتیبانی از چند رنگ و برخی حالت های جالب:

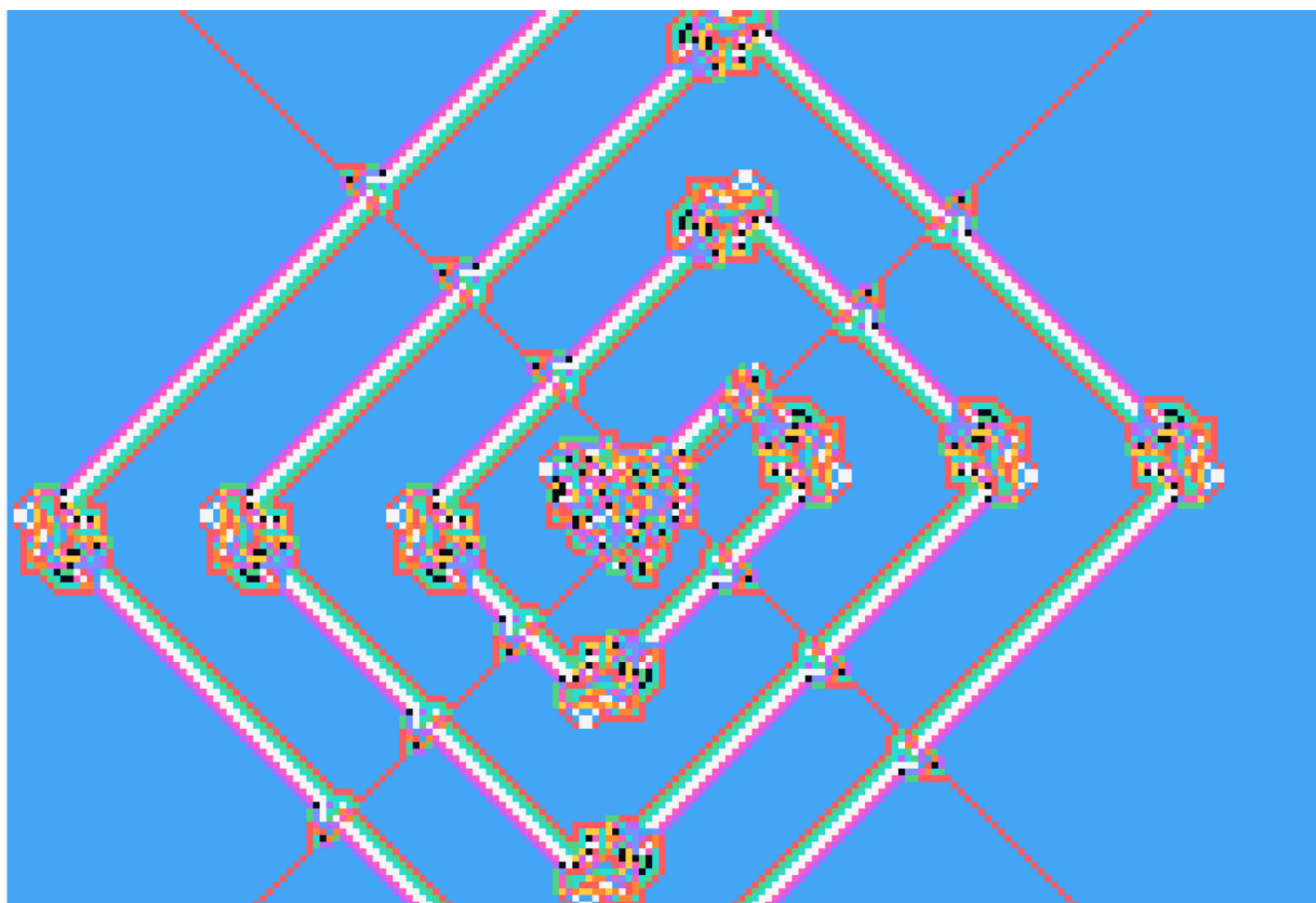
کاردیوید



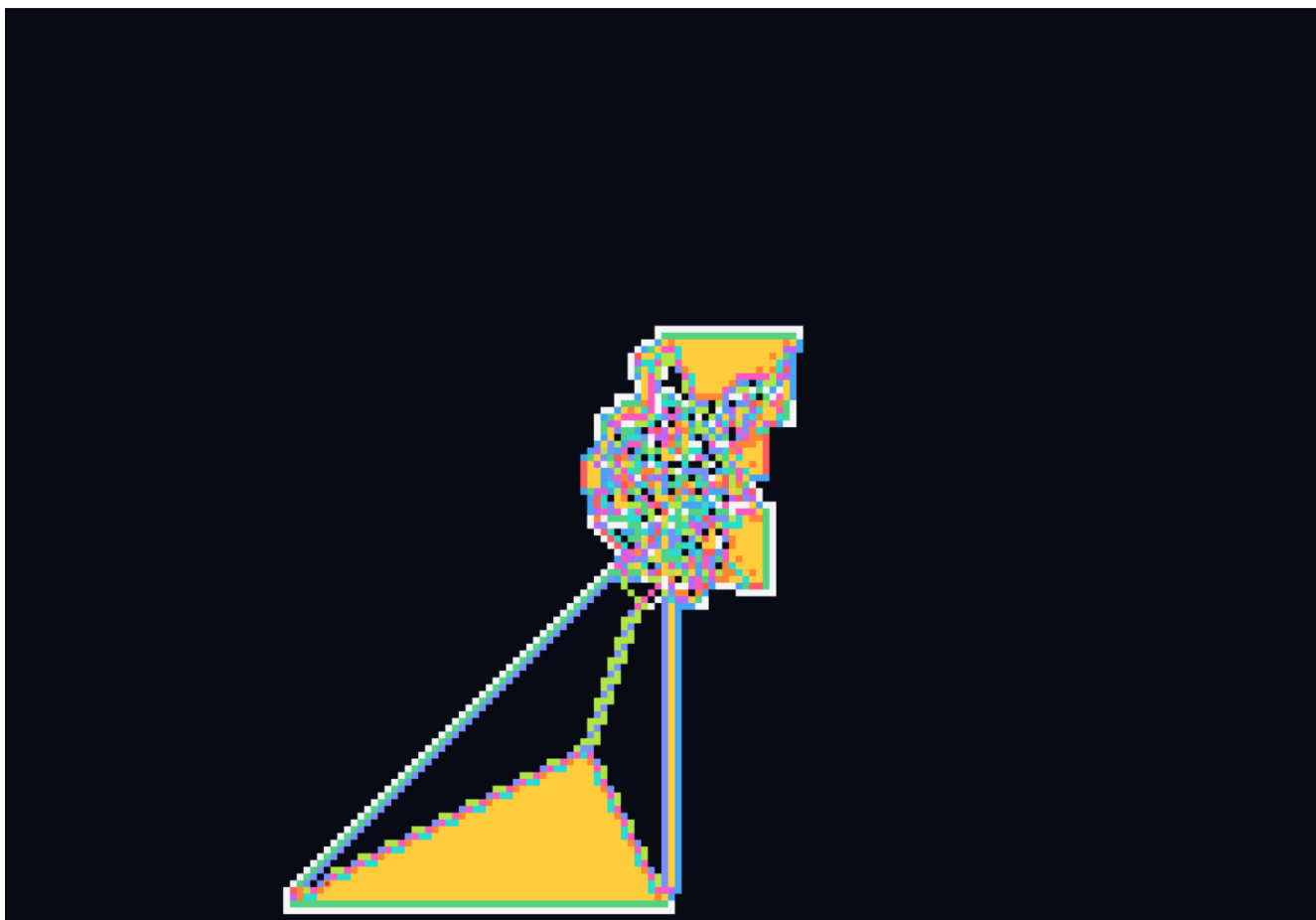
هزارتو(؟)ی متقارن:







مثث در حال رشد:



بخش سوم

پیاده‌سازی گیت منطقی AND با استفاده از گلایدرها:

هدف این بخش، پیاده‌سازی گیت منطقی AND با استفاده از حرکت و برخورد گلایدرها در بازی زندگی کانوی است. در این روش، هر گلايدر نقش یک سیگنال منطقی را دارد. وجود گلايدر در مسیر ورودی به معنی بیت ۱ و نبود گلايدر به معنی بیت ۰ در نظر گرفته می‌شود. در گیت AND، فقط زمانی خروجی ۱ خواهد بود که هر دو ورودی A و B دارای گلايدر باشند.

برای ساخت این گیت، دو گلايدر با جهت حرکت متفاوت در نظر گرفته شده‌اند. گلايدر اول مربوط به ورودی A است و در مسیر مورب رو به بالا و راست حرکت می‌کند. گلايدر دوم مربوط به ورودی B است و در مسیر مورب رو به پایین و راست حرکت می‌کند. این دو مسیر نسبت به هم عمود هستند و با تنظیم درست موقعیت اولیه و زمان شروع حرکت، گلايدها در یک نقطه‌ی مشخص به هم می‌رسند.

```
_NE_GLIDER = (  
    (0, 1),  
    (0, 2),  
    (1, 0),  
    (1, 2),  
    (2, 2),  
)  
  
_SE_GLIDER = (  
    (0, 1),  
    (1, 2),  
    (2, 0),  
    (2, 1),  
    (2, 2),  
)
```

در ابتدای کلاس، مختصات دو الگوی گلايدر در متغیرهای `_NE_GLIDER` و `_SE_GLIDER` ذخیره شده‌اند. این مختصات شکل اولیه‌ی دو گلايدر را مشخص می‌کنند. تفاوت این دو الگو در جهت حرکت آن‌ها است؛ یکی در جهت شمال شرقی و دیگری در جهت جنوب شرقی حرکت می‌کند. استفاده از دو جهت متفاوت باعث می‌شود که گلايدهای ورودی A و B بتوانند با زاویه‌ی ۹۰ درجه با هم برخورد کنند.

```

91     @staticmethod
92     def _new_life(grid_size):
93         if grid_size < 35:
94             raise ValueError("grid_size must be at least 35")
95         return GameOfLife(grid_size, finite=True, fastMode=True)
96
97     @staticmethod
98     def _place_glider(life, pattern, origin):
99         base_r, base_c = origin
100         for dr, dc in pattern:
101             life.grid[base_r + dr, base_c + dc] = life.aliveValue

```

متد `_new_life` یک نمونه‌ی جدید از کلاس `GameOfLife` می‌سازد. شبکه در این متد به‌صورت محدود یا `finite` ایجاد می‌شود. دلیل استفاده از شبکه‌ی محدود این است که گلایدری که از لبه‌ی صفحه خارج می‌شود، از سمت دیگر شبکه وارد نشود. اگر شبکه حالت حلقوی داشته باشد، گلایدرها ممکن است پس از خروج دوباره وارد مدار شوند و در نتیجه باعث ایجاد برخوردهای ناخواسته و اشتباه در خروجی گیت شوند.

متد `_place_glider` برای قرار دادن یک گلايدر در مختصات دلخواه استفاده می‌شود. این متد الگوی گلايدر و محل شروع آن را دریافت می‌کند. سپس مختصات نسبی سلول‌های زنده‌ی گلايدر را به مختصات مبدا اضافه می‌کند و سلول‌های مربوطه را در شبکه زنده قرار می‌دهد. با استفاده از این متد، می‌توان هر گلايدر را در مسیر مناسب ورودی `A` یا `B` قرار داد، بدون اینکه لازم باشد مختصات تمام سلول‌ها دوباره نوشته شوند.

```

217     def setup_and_gate(
218         self, grid_size=35, input_a_present=False, input_b_present=False
219     ):
220         life = self._new_life(grid_size)
221         if input_a_present:
222             self._place_glider(life, self._NE_GLIDER, (23, 2))
223         if input_b_present:
224             self._place_glider(life, self._SE_GLIDER, (3, 0))
225         return life
226

```

متد `setup_and_gate` وضعیت اولیه‌ی گیت `AND` را برای یک جفت ورودی آماده می‌کند. این متد دو ورودی بولی به نام‌های `input_a_present` و `input_b_present` می‌گیرد. اگر مقدار ورودی `A` برابر با `True` باشد، گلايدر مربوط به `A` در شبکه قرار داده می‌شود. اگر مقدار ورودی `B` برابر با `True` باشد، گلايدر مربوط به `B` در شبکه قرار می‌گیرد.

بنابراین، متد `setup_and_gate` چهار حالت ممکن ورودی را ایجاد می‌کند. در حالت `۰۰`، هیچ گلايدری وارد شبکه نمی‌شود. در حالت `۱۰`، فقط گلايدر `A` قرار می‌گیرد. در حالت `۰۱`، فقط گلايدر `B` قرار می‌گیرد. در حالت `۱۱`، هر دو گلايدر هم‌زمان در موقعیت‌های ورودی خود قرار داده می‌شوند تا بعد از چند نسل به نقطه‌ی برخورد برسند.

```

118     def _and_output_present(self, life):
119         r, c = self.AND_TARGET
120         return (
121             life.grid[r, c] == life.aliveValue
122             and life.grid[r, c + 1] == life.aliveValue
123             and life.grid[r + 1, c] == life.aliveValue
124             and life.grid[r + 1, c + 1] == life.aliveValue
125         )

```

متد `_and_output_present` برای بررسی خروجی گیت AND نوشته شده است. خروجی مورد انتظار گیت یک بلوک پایدار ۲ در ۲ است که در مختصات هدف قرار می‌گیرد. این متد چهار خانه‌ی مربوط به بلوک را بررسی می‌کند. اگر هر چهار خانه زنده باشند، یعنی ساختار بلوک تشکیل شده است و متد مقدار `True` را برمی‌گرداند. در غیر این صورت، مقدار `False` برگردانده می‌شود.

بلوک ۲ در ۲ یک ساختار پایدار در بازی زندگی کانوی است. هر سلول زنده‌ی این بلوک سه همسایه‌ی زنده دارد و به همین دلیل طبق قانون بقا زنده باقی می‌ماند. همچنین هیچ سلول مرده‌ای در اطراف آن دقیقاً سه همسایه‌ی زنده ندارد؛ بنابراین سلول جدیدی در اطراف بلوک متولد نمی‌شود. به همین دلیل، اگر گلايدر دیگری به بلوک نرسد، این ساختار بدون تغییر باقی می‌ماند.

```

233
234     def run_and_gate(self, input_a_present, input_b_present):
235         life = self.setup_and_gate(
236             input_a_present=input_a_present,
237             input_b_present=input_b_present,
238         )
239         for _ in range(self.AND_STEPS):
240             life.evolve()
241         return bool(self._and_output_present(life))
242

```

متد `run_and_gate` اجرای کامل گیت AND را انجام می‌دهد. ابتدا با استفاده از متد `setup_and_gate`، گلايدرهای مربوط به ورودی‌ها در شبکه قرار داده می‌شوند. سپس متد `evolve` به تعداد مشخصی نسل اجرا می‌شود تا گلايدرها حرکت کنند و در صورت وجود هر دو ورودی، به نقطه‌ی برخورد برسند.

پس از تکامل شبکه، متد `_and_output_present` فراخوانی می‌شود تا وجود یا نبود بلوک ۲ در ۲ در محل خروجی بررسی شود. نتیجه‌ی این بررسی به‌عنوان خروجی گیت برگردانده می‌شود. بنابراین، این متد نقش اصلی اجرای گیت برای یک جفت ورودی را بر عهده دارد.

توضیح عملکرد گیت در حالت‌های مختلف:

در حالت $A=0$ و $B=0$ هیچ گلایدری در شبکه قرار نمی‌گیرد. در نتیجه، هیچ برخوردی رخ نمی‌دهد و بلوک خروجی تشکیل نمی‌شود. بنابراین خروجی برابر با صفر است.

در حالت $A=1$ و $B=0$ فقط گلایدر A حرکت می‌کند. چون گلایدر B وجود ندارد، برخورد لازم برای ایجاد بلوک اتفاق نمی‌افتد. گلایدر A مسیر طبیعی خود را طی می‌کند و پس از رسیدن به مرز شبکه از صفحه خارج می‌شود. بنابراین خروجی صفر باقی می‌ماند.

در حالت $A=0$ و $B=1$ نیز فقط گلایدر B حرکت می‌کند. این گلایدر بدون برخورد مسیر خود را ادامه می‌دهد و از شبکه خارج می‌شود. در نتیجه، بلوک خروجی تشکیل نمی‌شود و خروجی صفر است.

در حالت $A=1$ و $B=1$ هر دو گلایدر وارد شبکه می‌شوند. به دلیل انتخاب دقیق موقعیت اولیه، این دو گلایدر در زمان مناسب و در محل مشخص با هم برخورد می‌کنند. نتیجه‌ی برخورد آن‌ها تشکیل یک بلوک پایدار ۲ در ۲ در ناحیه‌ی خروجی است. این بلوک نشان‌دهنده‌ی خروجی ۱ است.

```
131
132     def setup_and_stream(self, a_stream, b_stream, grid_size=35, period=None):
133         """Create an AND gate that accepts synchronized streams of input bits."""
134         a_bits = self._bits(a_stream)
135         b_bits = self._bits(b_stream)
136         if len(a_bits) != len(b_bits):
137             raise ValueError("AND input streams must have the same length")
138
139         period = self.STREAM_PERIOD if period is None else int(period)
140         if period < 44:
141             raise ValueError("period must be at least 44 generations")
142
143         life = self._new_life(grid_size)
144         injections = {}
145         samples = {}
146
147         for slot, (a_bit, b_bit) in enumerate(zip(a_bits, b_bits)):
148             launch_time = slot * period
149             packets = []
150             if a_bit:
151                 packets.append((self._NE_GLIDER, (23, 2)))
152             if b_bit:
153                 packets.append((self._SE_GLIDER, (3, 0)))
154             injections.setdefault(launch_time, []).extend(packets)
155
156             sample_time = launch_time + self.AND_STEPS
157             samples[sample_time] = slot
158
159         return _StreamingCircuit(
160             life,
161             injections,
162             samples,
163             self._and_output_present,
164         )
165
```

علاوه بر اجرای گیت برای یک جفت ورودی، متد `setup_and_stream` برای پردازش جریان بیت‌ها اضافه شده است. در این روش، به جای دریافت تنها یک بیت برای `A` و `B`، دو رشته‌ی باینری مانند `۱۰۱۱۰۱` و `۱۱۰۰۱۱` دریافت می‌شود. هر موقعیت از رشته، یک بازه‌ی زمانی مشخص از جریان داده را نشان می‌دهد.

اگر بیت مربوط به `A` یا `B` برابر با ۱ باشد، گلايدر آن ورودی در بازه‌ی زمانی مربوطه وارد شبکه می‌شود. اگر بیت برابر با صفر باشد، هیچ گلايدری برای آن ورودی شلیک نمی‌شود. فاصله‌ی زمانی بین بسته‌های متوالی به‌صورت ۴۸ نسل در نظر گرفته شده است تا گلايدرهای مربوط به بیت‌های مختلف با یکدیگر تداخل نداشته باشند.

```
196
197 def run_and_stream(self, a_stream, b_stream, period=None):
198     """Return the AND result for every synchronized input time slot."""
199     a_bits = self._bits(a_stream)
200     circuit = self.setup_and_stream(a_bits, b_stream, period=period)
201     actual_period = self.STREAM_PERIOD if period is None else int(period)
202     end = (len(a_bits) - 1) * actual_period + self.AND_STEPS
203     return self._run(circuit, end)
204
```

متد `run_and_stream` گیت AND را برای کل جریان ورودی اجرا می‌کند. ابتدا جریان ورودی به بیت‌های منطقی تبدیل می‌شود، سپس مدار جریان‌محور ساخته می‌شود و شبیه‌سازی تا زمان لازم اجرا می‌گردد. در هر بازه‌ی زمانی مشخص، محل خروجی بررسی می‌شود و نتیجه‌ی آن بیت در خروجی ذخیره می‌شود.

برای نمونه، اگر ورودی‌ها به شکل `A = ۱۰۱۱۰۱` و `B = ۱۱۰۰۱۱` باشند، نتیجه‌ی AND به‌صورت `۱۰۰۰۰۱` خواهد بود. این خروجی نشان می‌دهد که فقط در موقعیت‌هایی که هر دو جریان دارای بیت ۱ هستند، خروجی ۱ تولید شده است.

در مجموع، این پیاده‌سازی نشان می‌دهد که برخورد دقیق گلايدرها می‌تواند نقش یک عملگر منطقی را ایفا کند. در این گیت، ساختار بلوک ۲ در ۲ معادل خروجی ۱ و نبود آن معادل خروجی ۰ است. تشکیل این ساختار تنها در صورت حضور هم‌زمان هر دو گلايدر ورودی، رفتار جدول درستی گیت AND را پیاده‌سازی می‌کند.

پیاده‌سازی گیت منطقی NOT :

در این پیاده‌سازی، حضور یا نبود گلايدر در یک بازه‌ی زمانی مشخص به‌ترتیب معادل بیت یک و بیت صفر در نظر گرفته شده است. برخلاف گیت AND که برای تشکیل خروجی به حضور هم‌زمان دو گلايدر ورودی نیاز داشت، در گیت NOT یک جریان کنترل دائمی وجود دارد. این جریان کنترل به‌صورت طبیعی توسط تفنگ گلايدر Gosper تولید می‌شود و نقش پایه‌ی خروجی را بر عهده دارد.

توضیح ساختار جریان کنترل در گیت NOT

برای تولید جریان کنترل، از متد insertGliderGun استفاده شده است. این متد الگوی Gosper Glider Gun را در شبکه قرار می‌دهد. تفنگ گلايدر یک ساختار نوسانی است که پس از شروع شبیه‌سازی، به‌صورت دوره‌ای و بدون نیاز به ایجاد دستی گلايدر جدید، گلايدر تولید می‌کند. در این مدار، تفنگ گلايدر هر ۳۰ نسل یک گلايدر کنترل تولید می‌کند.

استفاده از تفنگ گلايدر باعث می‌شود جریان کنترل کاملاً بخشی از جهان بازی زندگی باشد. پس از قرار گرفتن الگوی اولیه‌ی تفنگ در شبکه، تمام گلايدهای کنترل فقط در نتیجه‌ی قوانین چهارگانه‌ی بازی زندگی تولید می‌شوند. بنابراین در طول اجرای شبیه‌سازی، هیچ گلايدر کنترلی به‌صورت مستقیم یا مصنوعی داخل شبکه قرار داده نمی‌شود.

برای ایجاد فضای کافی برای تفنگ گلايدر، مسیر حرکت گلايدها، محل برخورد و ناحیه‌ی خروجی، اندازه‌ی شبکه‌ی گیت NOT بزرگ‌تر از گیت AND در نظر گرفته شده است. شبکه در حالت محدود اجرا می‌شود تا گلايدهایی که از مدار خارج می‌شوند، از سمت دیگر صفحه وارد مدار نشوند و باعث برخوردهای ناخواسته نشوند.

توضیح ثابت‌های مربوط به گیت NOT

در کد، مختصات تفنگ گلايدر، زمان رسیدن اولین گلايدر کنترل به مسیر موردنظر، فاصله‌ی زمانی بین گلايدهای تولیدشده و مختصات ناحیه‌ی خروجی مشخص شده‌اند. این مقادیر برای هماهنگ کردن گلايدر ورودی با گلايدر کنترل لازم هستند.

تفنگ گلايدر در مختصات مشخصی از شبکه قرار می‌گیرد. اولین گلايدر کنترل پس از تعداد مشخصی نسل به ناحیه‌ی برخورد می‌رسد و گلايدهای بعدی نیز با فاصله‌ی ۳۰ نسل از یکدیگر وارد همان مسیر می‌شوند. به همین دلیل، هر بیت از استریم ورودی با یکی از گلايدهای تولیدشده توسط تفنگ هماهنگ می‌شود.

```
126
127 def _not_output_present(self, life):
128     return all(
129         life.grid[r, c] == life.aliveValue for r, c in self.NOT_OUTPUT_CELLS
130     )
```

متد `not_output_present` وظیفه دارد خروجی گیت NOT را بررسی کند. خروجی گیت در این پیاده‌سازی، یک گلايدر کنترل است که پس از عبور از ناحیه‌ی برخورد، در محل مشخصی از شبکه ظاهر می‌شود. این متد مختصات پنج سلول زنده‌ی گلايدر را در ناحیه‌ی خروجی بررسی می‌کند.

اگر هر پنج سلول مربوط به الگوی گلايدر زنده باشند، یعنی گلايدر کنترل سالم از ناحیه‌ی برخورد عبور کرده است و خروجی گیت برابر با یک در نظر گرفته می‌شود. اگر این الگو در خروجی وجود نداشته باشد، یعنی گلايدر کنترل در برخورد با گلايدر ورودی نابود شده است و خروجی گیت برابر با صفر خواهد بود.

```
165
166 def setup_not_stream(self, a_stream, grid_size=120, period=None):
167     """Create a NOT gate driven by a real Gosper-gun control stream."""
168     a_bits = self._bits(a_stream)
169
170     # The Gosper gun and its collision/output lanes need a larger board.
171     life = self._new_life(max(grid_size, 120))
172     life.insertGliderGun(self.NOT_GUN_ORIGIN)
173
174     injections = {}
175     samples = {}
176
177     for slot, a_bit in enumerate(a_bits):
178         control_time = self.NOT_GUN_FIRST_SLOT + slot * self.NOT_GUN_PERIOD
179
180         # The control bit is emitted naturally by the gun. Only an input
181         # value of 1 launches an artificial NE glider. Its collision with
182         # the corresponding control glider annihilates both of them.
183         if a_bit:
184             injections[control_time] = [
185                 (self._NE_GLIDER, self.NOT_INPUT_ORIGIN)
186             ]
187
188         samples[control_time + self.NOT_STEPS] = slot
189
190     return _StreamingCircuit(
191         life,
192         injections,
193         samples,
194         self._not_output_present,
195     )
196
```

متد `setup_not_stream` وضعیت اولیه‌ی مدار NOT را برای یک جریان ورودی آماده می‌کند. ابتدا استریم ورودی دریافت شده و هر کاراکتر آن به یک مقدار منطقی تبدیل می‌شود. سپس یک شبکه‌ی جدید ساخته شده و تفنگ گلايدر Gosper در آن قرار می‌گیرد. در ادامه، برای هر بیت از استریم ورودی، زمان رسیدن گلايدر کنترل متناظر تعیین می‌شود. اگر بیت ورودی برابر با صفر باشد، هیچ گلايدری از سمت ورودی شلیک نمی‌شود. در نتیجه، گلايدر کنترل تولیدشده توسط تفنگ بدون برخورد حرکت می‌کند و به ناحیه‌ی خروجی می‌رسد.

اگر بیت ورودی برابر با یک باشد، یک گلايدر ورودی در زمان مشخصی در شبکه قرار داده می‌شود. مسیر و زمان‌بندی این گلايدر طوری انتخاب شده است که هم‌زمان با گلايدر کنترل متناظر برخورد کند. در این برخورد، گلايدر ورودی و گلايدر کنترل هر دو نابود می‌شوند. بنابراین گلايدر کنترل به خروجی نمی‌رسد و آن بازه‌ی زمانی در خروجی برابر با صفر خواهد بود.

نکته‌ی مهم این است که گلايدر ورودی فقط در زمان‌هایی به‌صورت مصنوعی وارد شبکه می‌شود که بیت استریم ورودی برابر با یک باشد. اما جریان کنترل در تمام زمان‌ها توسط تفنگ گلايدر و به‌صورت طبیعی تولید می‌شود. به این ترتیب، ورودی A نقش سیگنال حذف‌کننده را دارد و جریان کنترل نقش خروجی پیش‌فرض را ایفا می‌کند.

```
204
205     def run_not_stream(self, a_stream, period=None):
206         """Return the NOT result for every Gosper-gun output time slot."""
207         a_bits = self._bits(a_stream)
208         circuit = self.setup_not_stream(a_bits)
209         end = (
210             self.NOT_GUN_FIRST_SLOT
211             + (len(a_bits) - 1) * self.NOT_GUN_PERIOD
212             + self.NOT_STEPS
213         )
214         return self._run(circuit, end)
215
```

متد `run_not_stream` گیت NOT را برای کل جریان ورودی اجرا می‌کند. ابتدا مدار توسط `setup_not_stream` ساخته می‌شود. سپس شبیه‌سازی تا زمانی ادامه پیدا می‌کند که گلايدرهای کنترل مربوط به تمام بیت‌های ورودی فرصت رسیدن به ناحیه‌ی خروجی را داشته باشند.

در هر بازه‌ی زمانی مربوط به یک بیت، وضعیت خروجی با استفاده از متد `not_output_present` بررسی می‌شود. نتیجه‌ی هر بررسی در یک لیست ذخیره شده و در پایان، خروجی کامل استریم NOT تولید می‌شود. طول خروجی برابر با طول استریم ورودی است، اما مقدار هر بیت معکوس بیت متناظر در ورودی خواهد بود.

برای نمونه، اگر استریم ورودی برابر با ۱۰۱۱۰۱ باشد، در موقعیت‌های دارای بیت یک، گلايدر ورودی با گلايدر کنترل برخورد می‌کند و خروجی صفر می‌شود. در موقعیت‌های دارای بیت صفر، گلايدر کنترل بدون برخورد حرکت می‌کند و خروجی یک می‌شود. بنابراین خروجی نهایی برابر با ۰۱۰۰۱۰ خواهد بود.

```

226
227     def setup_not_gate(self, grid_size=120, input_a_present=False):
228         # Keep the template method, but build its control signal with a real
229         # Gosper gun instead of inserting a control glider directly.
230         return self.setup_not_stream(
231             [input_a_present], grid_size=max(grid_size, 120)
232         )
233

```

متد `setup_not_gate` همان ساختار موردنیاز تمپلیت پروژه را حفظ می‌کند و برای اجرای گیت NOT با یک ورودی استفاده می‌شود. این متد در واقع نسخه‌ی تک‌بیتی گیت NOT را از طریق متد `setup_not_stream` ایجاد می‌کند. تفاوت مهم این نسخه با پیاده‌سازی اولیه این است که جریان کنترل با یک گلايدر قرارداده‌شده‌ی دستی ساخته نمی‌شود، بلکه تفنگ گلايدر Gosper آن را تولید می‌کند.

```

242
243     def run_not_gate(self, input_a_present):
244         circuit = self.setup_not_gate(input_a_present=input_a_present)
245         end = self.NOT_GUN_FIRST_SLOT + self.NOT_STEPS
246         output = self._run(circuit, end)
247         return bool(output[0])
248

```

متد `run_not_gate` نسخه‌ی تک‌بیتی گیت NOT را اجرا می‌کند. ابتدا مدار برای مقدار ورودی ساخته می‌شود، سپس شبیه‌سازی تا رسیدن گلايدر کنترل به ناحیه‌ی خروجی ادامه پیدا می‌کند. در پایان، وجود یا نبود گلايدر در خروجی بررسی می‌شود و نتیجه به صورت مقدار منطقی بازگردانده می‌شود.

اگر ورودی صفر باشد، هیچ گلايدر ورودی برای برخورد با جریان کنترل وجود ندارد. در نتیجه، گلايدر تولیدشده توسط تفنگ به خروجی می‌رسد و مقدار خروجی یک می‌شود. اگر ورودی یک باشد، گلايدر ورودی با گلايدر کنترل برخورد می‌کند و هر دو از بین می‌روند. در نتیجه، گلايدری در خروجی مشاهده نمی‌شود و مقدار خروجی صفر خواهد بود.

در مجموع، گیت NOT با استفاده از یک جریان دائمی گلايدر که توسط Gosper Glider Gun تولید می‌شود و برخورد کنترل‌شده‌ی گلايدر ورودی با این جریان پیاده‌سازی شده است. این ساختار رفتار منطقی نقیض را به‌درستی نشان می‌دهد، زیرا نبود سیگنال ورودی باعث عبور سیگنال کنترل و تولید خروجی یک می‌شود، در حالی که حضور سیگنال ورودی باعث نابودی سیگنال کنترل و تولید خروجی صفر خواهد شد.